

# AssetTrack — API Design

**Base URL:** `http://localhost:8080/api/v1`  
**Auth:** Bearer JWT in `Authorization` header  
**Content-Type:** `application/json`

## Standard Response Envelope

Every API response — success or failure — is wrapped in a consistent envelope. Clients should always check `success` first, then read either `data` or `error`.

### Success Wrapper

```
{
  "success": true,
  "status": 200,
  "message": "Operation completed successfully.",
  "data": { }
}
```

| Field                | Type                  | Description                                                                       |
|----------------------|-----------------------|-----------------------------------------------------------------------------------|
| <code>success</code> | boolean               | Always <code>true</code> for successful responses                                 |
| <code>status</code>  | int                   | HTTP status code mirrored in the body                                             |
| <code>message</code> | string                | Human-readable summary of the result                                              |
| <code>data</code>    | object   array   null | The actual response payload; <code>null</code> for operations that return nothing |

For paginated responses, `data` contains a `pagination` sub-object alongside `items`:

```
{
  "success": true,
  "status": 200,
  "message": "Assets retrieved successfully.",
  "data": {
    "items": [ ],
    "pagination": {
      "page": 0,
      "size": 20,
      "totalElements": 87,
      "totalPages": 5
    }
  }
}
```

```
    "totalPages": 5
  }
}
}
```

## Error Wrapper

```
{
  "success": false,
  "status": 404,
  "message": "Asset with id 'a-999' not found.",
  "error": {
    "code": "ASSET_NOT_FOUND",
    "details": []
  }
}
```

| Field         | Type    | Description                                                              |
|---------------|---------|--------------------------------------------------------------------------|
| success       | boolean | Always false for error responses                                         |
| status        | int     | HTTP status code mirrored in the body                                    |
| message       | string  | Human-readable description of the error                                  |
| error.code    | string  | Machine-readable error code for client logic                             |
| error.details | array   | Field-level validation errors (populated on 400 / 422 , empty otherwise) |

For validation errors, `error.details` is an array of field-level problems:

```
{
  "success": false,
  "status": 422,
  "message": "Validation failed. Please check the submitted fields.",
  "error": {
    "code": "VALIDATION_ERROR",
    "details": [
      { "field": "email", "issue": "must be a valid email address" },
      { "field": "password", "issue": "must be at least 8 characters" }
    ]
  }
}
```

## Error Code Reference

| HTTP | Error Code | Meaning |
|------|------------|---------|
|------|------------|---------|

| Status | Error Code       | Meaning                                                            |
|--------|------------------|--------------------------------------------------------------------|
| 400    | BAD_REQUEST      | Malformed request body or missing required fields                  |
| 401    | UNAUTHORIZED     | Missing or expired JWT                                             |
| 403    | FORBIDDEN        | Authenticated but insufficient role                                |
| 404    | *_NOT_FOUND      | Resource does not exist (e.g. ASSET_NOT_FOUND , USER_NOT_FOUND )   |
| 409    | *_CONFLICT       | Duplicate resource (e.g. SERIAL_NUMBER_CONFLICT , EMAIL_CONFLICT ) |
| 422    | VALIDATION_ERROR | Request body failed validation rules                               |
| 500    | INTERNAL_ERROR   | Unexpected server-side failure                                     |

---

## Authentication

---

### POST /auth/register

Register a new user.

#### Request:

```
{
  "name": "Jane Doe",
  "email": "jane@company.com",
  "password": "StrongP@ss1"
}
```

#### Response 201 Created :

```
{
  "success": true,
  "status": 201,
  "message": "User registered successfully.",
  "data": {
    "id": "u-001",
    "name": "Jane Doe",
    "email": "jane@company.com",
    "role": "DEVELOPER",
    "createdAt": "2025-06-01T09:00:00Z"
  }
}
```

## Error — email already exists 409 :

```
{
  "success": false,
  "status": 409,
  "message": "An account with this email already exists.",
  "error": {
    "code": "EMAIL_CONFLICT",
    "details": []
  }
}
```

---

## POST /auth/login

Authenticate and receive a JWT.

### Request:

```
{
  "email": "jane@company.com",
  "password": "StrongP@ss1"
}
```

### Response 200 OK :

```
{
  "success": true,
  "status": 200,
  "message": "Login successful.",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs...\"",
    "tokenType": "Bearer",
    "expiresIn": 3600,
    "user": {
      "id": "u-001",
      "name": "Jane Doe",
      "role": "DEVELOPER"
    }
  }
}
```

## Error — invalid credentials 401 :

```
{
  "success": false,
  "status": 401.
}
```

```
{
  "message": "Invalid email or password.",
  "error": {
    "code": "UNAUTHORIZED",
    "details": []
  }
}
```

---

## Users

---

All endpoints require authentication. Role management requires `ADMIN`.

### GET `/users`

List all users with optional search filters.

**Roles:** `ADMIN`, `MANAGER`

**Query params:**

- `page` (int, default `0`): zero-based page number
- `size` (int, default `20`): items per page (max `100`)
- `role` (string, optional): filter by role ( `ADMIN`, `MANAGER`, `DEVELOPER` )
- `q` (string, optional): unified search across all user fields (name, email, ID) — uses PostgreSQL trigram similarity for fuzzy matching + substring match, case-insensitive

When `role` and `q` are both provided, results match BOTH filters (AND logic). Results are ranked by relevance (similarity score) when search is used.

**Example requests:**

- `GET /users?page=0&size=10` — list first 10 users
- `GET /users?role=MANAGER` — list all managers
- `GET /users?q=Jane&role=DEVELOPER` — find developers matching "Jane" (name, email, or ID)
- `GET /users?q=company.com` — find users with "company.com" in their email

**Response `200 OK`:**

```
{
  "success": true,
  "status": 200,
  "message": "Users retrieved successfully.",
  "data": {
    "items": [
      {
        "id": "u-001",
```

```
      "name": "Jane Doe",
      "email": "jane@company.com",
      "role": "DEVELOPER",
      "createdAt": "2025-06-01T09:00:00Z"
    }
  ],
  "pagination": {
    "page": 0,
    "size": 20,
    "totalElements": 42,
    "totalPages": 3
  }
}
```

---

## GET /users/{id}

Get a single user by ID.

**Roles:** ADMIN, MANAGER, or self

**Response 200 OK :**

```
{
  "success": true,
  "status": 200,
  "message": "User retrieved successfully.",
  "data": {
    "id": "u-001",
    "name": "Jane Doe",
    "email": "jane@company.com",
    "role": "DEVELOPER",
    "assignedAssets": ["a-010", "a-022"],
    "createdAt": "2025-06-01T09:00:00Z"
  }
}
```

**Error — not found 404 :**

```
{
  "success": false,
  "status": 404,
  "message": "User with id 'u-999' not found.",
  "error": {
    "code": "USER_NOT_FOUND",
    "details": []
  }
}
```

---

## PATCH /users/{id}/role

Update a user's role.

**Roles:** ADMIN only

**Request:**

```
{
  "role": "MANAGER"
}
```

**Response 200 OK :**

```
{
  "success": true,
  "status": 200,
  "message": "User role updated successfully.",
  "data": {
    "id": "u-001",
    "name": "Jane Doe",
    "role": "MANAGER"
  }
}
```

---

## DELETE /users/{id}

Delete a user account.

**Roles:** ADMIN only

**Response 200 OK :**

```
{
  "success": true,
  "status": 200,
  "message": "User deleted successfully.",
  "data": null
}
```

---

## Assets

ADMIN and MANAGER can create/update/delete. DEVELOPER can view their own

assets and submit condition reports.

## GET /assets

List all assets with optional filters.

**Roles:** ADMIN , MANAGER

**Query params:**

| Param                 | Type             | Description                                                                                                                                                                        |
|-----------------------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| type                  | string           | LAPTOP , MONITOR , ACCESSORY                                                                                                                                                       |
| status                | string           | AVAILABLE , ASSIGNED , UNDER_REPAIR , DECOMMISSIONED                                                                                                                               |
| q                     | string, optional | Unified search across all asset fields (id, type, brand, model, serial number, status) — uses PostgreSQL trigram similarity for fuzzy matching + substring match, case-insensitive |
| assignedUserId        | string           | Filter by owner (user UUID)                                                                                                                                                        |
| warrantyExpiresBefore | date             | ISO 8601 date                                                                                                                                                                      |
| page                  | int              | Page number (default 0)                                                                                                                                                            |
| size                  | int              | Page size (default 20)                                                                                                                                                             |

When multiple filters are provided, results match ALL filters (AND logic). Results are ranked by relevance (similarity score) when search is used.

### Example requests:

- GET /assets?page=0&size=20 — list first 20 assets
- GET /assets?type=LAPTOP&status=AVAILABLE — find available laptops
- GET /assets?q=Dell — find assets matching "Dell" (brand, model, serial number, type, status, or id)
- GET /assets?q=LAPTOP — find assets by type
- GET /assets?q=AVAILABLE — find assets by status
- GET /assets?q=XPS&type=LAPTOP — find laptops matching "XPS"
- GET /assets?q=SN-XPS-0042 — find asset by serial number
- GET /assets?q=a-010 — find asset by id

**Response 200 OK :**

```
{
  "success": true,
  "status": 200,
  "message": "Assets retrieved successfully.",
  "data": {
    "items": [
      {
        "id": "a-010",
        "type": "LAPTOP",
        "brand": "Dell",
        "model": "XPS 15",
        "serialNumber": "SN-XPS-0042",
        "status": "ASSIGNED",
        "assignedTo": {
          "id": "u-001",
          "name": "Jane Doe"
        },
        "purchaseDate": "2023-03-15",
        "warrantyExpirationDate": "2026-03-15",
        "warrantyExpired": false
      }
    ],
    "pagination": {
      "page": 0,
      "size": 20,
      "totalElements": 87,
      "totalPages": 5
    }
  }
}
```

---

## POST /assets

Register a new asset.

**Roles:** ADMIN, MANAGER

**Request:**

```
{
  "type": "LAPTOP",
  "brand": "Dell",
  "model": "XPS 15",
  "serialNumber": "SN-XPS-0042",
  "purchaseDate": "2023-03-15",
  "warrantyExpirationDate": "2026-03-15"
}
```

**Response** 201 Created :

---

```
{
  "success": true,
  "status": 201,
  "message": "Asset registered successfully.",
  "data": {
    "id": "a-010",
    "type": "LAPTOP",
    "brand": "Dell",
    "model": "XPS 15",
    "serialNumber": "SN-XPS-0042",
    "status": "AVAILABLE",
    "purchaseDate": "2023-03-15",
    "warrantyExpirationDate": "2026-03-15",
    "createdAt": "2025-06-01T10:00:00Z"
  }
}
```

### Error — duplicate serial number 409 :

```
{
  "success": false,
  "status": 409,
  "message": "An asset with serial number 'SN-XPS-0042' already exists.",
  "error": {
    "code": "SERIAL_NUMBER_CONFLICT",
    "details": []
  }
}
```

If the database raises a duplicate constraint error during asset creation, the API returns the same 409 conflict response with a friendly serial-number message.

### Error — unexpected failure 500 :

```
{
  "success": false,
  "status": 500,
  "message": "An unexpected server error occurred. Please try again later.",
  "error": {
    "code": "INTERNAL_ERROR",
    "details": []
  }
}
```

---

## GET /assets/{id}

Get a single asset's full details

Get a single asset's full details.

**Roles:** ADMIN , MANAGER , DEVELOPER (own assets only)

**Response 200 OK :**

```
{
  "success": true,
  "status": 200,
  "message": "Asset retrieved successfully.",
  "data": {
    "id": "a-010",
    "type": "LAPTOP",
    "brand": "Dell",
    "model": "XPS 15",
    "serialNumber": "SN-XPS-0042",
    "status": "ASSIGNED",
    "assignedTo": {
      "id": "u-001",
      "name": "Jane Doe"
    },
    "purchaseDate": "2023-03-15",
    "warrantyExpirationDate": "2026-03-15",
    "warrantyExpired": false,
    "allocationHistory": [
      {
        "userId": "u-099",
        "userName": "Bob Smith",
        "assignedAt": "2023-03-20T08:00:00Z",
        "returnedAt": "2024-01-10T17:00:00Z"
      }
    ],
    "conditionReports": []
  }
}
```

**Error — not found 404 :**

```
{
  "success": false,
  "status": 404,
  "message": "Asset with id 'a-999' not found.",
  "error": {
    "code": "ASSET_NOT_FOUND",
    "details": []
  }
}
```

---

**PUT /assets/{id}**

Update asset metadata.

**Roles:** ADMIN , MANAGER

**Request:**

```
{
  "brand": "Dell",
  "model": "XPS 15 9500",
  "warrantyExpirationDate": "2027-03-15",
  "status": "UNDER_REPAIR"
}
```

**Response 200 OK :**

```
{
  "success": true,
  "status": 200,
  "message": "Asset updated successfully.",
  "data": {
    "id": "a-010",
    "brand": "Dell",
    "model": "XPS 15 9500",
    "warrantyExpirationDate": "2027-03-15",
    "status": "UNDER_REPAIR"
  }
}
```

---

## DELETE /assets/{id}

Decommission/delete an asset.

**Roles:** ADMIN only

**Response 200 OK :**

```
{
  "success": true,
  "status": 200,
  "message": "Asset decommissioned successfully.",
  "data": null
}
```

---

## GET /assets/spare-laptops

Find available spare laptops

and available spare laptops.

**Roles:** All authenticated users

**Response** 200 OK :

```
{
  "success": true,
  "status": 200,
  "message": "Available spare laptops retrieved successfully.",
  "data": {
    "items": [
      {
        "id": "a-055",
        "brand": "Lenovo",
        "model": "ThinkPad X1",
        "serialNumber": "SN-TP-0088",
        "status": "AVAILABLE",
        "lastOwner": {
          "id": "u-012",
          "name": "Alice Brown"
        },
        "warrantyExpirationDate": "2027-01-01"
      }
    ]
  }
}
```

---

## Asset Allocation

---

### POST /assets/{id}/allocations

Assign an asset to a user.

**Roles:** ADMIN , MANAGER

**Request:**

```
{
  "userId": "u-001"
}
```

**Response** 200 OK :

```
{
  "success": true,
  "status": 200,
  "message": "Asset assigned to user successfully.",
  "data": {
```

```
"data": {
  "assetId": "a-010",
  "assignedTo": {
    "id": "u-001",
    "name": "Jane Doe"
  },
  "assignedAt": "2025-06-01T11:00:00Z"
}
```

**Error — asset already assigned 409 :**

```
{
  "success": false,
  "status": 409,
  "message": "Asset 'a-010' is already assigned to another user.",
  "error": {
    "code": "ASSET_ALREADY_ASSIGNED",
    "details": []
  }
}
```

---

## DELETE /assets/{id}/allocations

Unassign (return) an asset.

**Roles:** ADMIN , MANAGER

**Response 200 OK :**

```
{
  "success": true,
  "status": 200,
  "message": "Asset returned and marked as available.",
  "data": {
    "assetId": "a-010",
    "status": "AVAILABLE",
    "returnedAt": "2025-06-15T17:00:00Z"
  }
}
```

---

## GET /assets/{id}/allocations

Get the full allocation history of an asset.

**Roles:** ADMIN , MANAGER

Response **200 OK** :

```
{
  "success": true,
  "status": 200,
  "message": "Allocation history retrieved successfully.",
  "data": {
    "items": [
      {
        "id": "al-001",
        "user": { "id": "u-099", "name": "Bob Smith" },
        "assignedAt": "2023-03-20T08:00:00Z",
        "returnedAt": "2024-01-10T17:00:00Z"
      },
      {
        "id": "al-002",
        "user": { "id": "u-001", "name": "Jane Doe" },
        "assignedAt": "2024-01-15T09:00:00Z",
        "returnedAt": null
      }
    ]
  }
}
```

---

## Condition Reports

---

### POST **/assets/{id}/condition-reports**

Submit a condition/issue report for an asset.

**Roles:** **DEVELOPER** (own assets), **ADMIN** , **MANAGER**

**Request:**

```
{
  "issue": "Battery drains within 2 hours of full charge",
  "severity": "HIGH"
}
```

**Severity values:** **LOW** , **MEDIUM** , **HIGH**

Response **201 Created** :

```
{
  "success": true,
  "status": 201,
  "message": "Condition report submitted successfully.",
}
```

```
"data": {
  "id": "cr-001",
  "assetId": "a-010",
  "reportedBy": { "id": "u-001", "name": "Jane Doe" },
  "issue": "Battery drains within 2 hours of full charge",
  "severity": "HIGH",
  "status": "OPEN",
  "createdAt": "2025-06-10T14:30:00Z"
}
```

#### Error — validation failure 422 :

```
{
  "success": false,
  "status": 422,
  "message": "Validation failed. Please check the submitted fields.",
  "error": {
    "code": "VALIDATION_ERROR",
    "details": [
      { "field": "severity", "issue": "must be one of: LOW, MEDIUM, HIGH" },
      { "field": "issue", "issue": "must not be blank" }
    ]
  }
}
```

---

## GET /condition-reports

List all condition reports.

**Roles:** ADMIN , MANAGER

**Query params:** status ( OPEN , IN\_PROGRESS , RESOLVED ), severity , assetId , page , size

#### Response 200 OK :

```
{
  "success": true,
  "status": 200,
  "message": "Condition reports retrieved successfully.",
  "data": {
    "items": [
      {
        "id": "cr-001",
        "assetId": "a-010",
        "reportedBy": { "id": "u-001", "name": "Jane Doe" },
        "issue": "Battery drains within 2 hours of full charge",
        "severity": "HIGH",
```

```
        "status": "OPEN",
        "createdAt": "2025-06-10T14:30:00Z"
      }
    ],
    "pagination": {
      "page": 0,
      "size": 20,
      "totalElements": 12,
      "totalPages": 1
    }
  }
}
```

---

## PATCH /condition-reports/{id}

Update the status of a condition report.

**Roles:** ADMIN, MANAGER

**Request:**

```
{
  "status": "RESOLVED",
  "resolution": "Replaced battery. Asset returned to user."
}
```

**Response 200 OK:**

```
{
  "success": true,
  "status": 200,
  "message": "Condition report updated successfully.",
  "data": {
    "id": "cr-001",
    "status": "RESOLVED",
    "resolution": "Replaced battery. Asset returned to user.",
    "resolvedAt": "2025-06-12T10:00:00Z"
  }
}
```

---

## Search

### GET /assets/search

Advanced asset search across multiple fields.

Advanced asset search across multiple fields:

**Roles:** ADMIN , MANAGER

### Query params:

| Param           | Type    | Description                                                                                                                                                        |
|-----------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| q               | string  | Free-text search across all asset fields (id, type, brand, model, serial number, status) — uses PostgreSQL trigram similarity for fuzzy matching + substring match |
| type            | string  | Asset type ( LAPTOP , MONITOR , ACCESSORY )                                                                                                                        |
| status          | string  | Asset status ( AVAILABLE , ASSIGNED , UNDER_REPAIR , DECOMMISSIONED )                                                                                              |
| assignedUserId  | string  | Owner user ID (UUID)                                                                                                                                               |
| brand           | string  | Brand name (legacy parameter, use q instead)                                                                                                                       |
| warrantyExpired | boolean | Filter expired warranties                                                                                                                                          |
| page            | int     | Page number (default 0)                                                                                                                                            |
| size            | int     | Page size (default 20)                                                                                                                                             |

### Example requests:

- GET /api/v1/assets/search?q=XPS&type=LAPTOP&status=AVAILABLE — find available laptops matching "XPS"
- GET /api/v1/assets/search?q=LAPTOP — find all laptops by type
- GET /api/v1/assets/search?q=a-010 — find asset by ID

### Response 200 OK :

```
{
  "success": true,
  "status": 200,
  "message": "Search completed successfully.",
  "data": {
    "items": [],
    "pagination": {
      "page": 0,
      "size": 20,
      "totalElements": 3,
      "totalPages": 1
    }
  }
}
```

---

## Reporting & Analytics

---

### GET /reports/dashboard

Aggregate dashboard statistics.

**Roles:** ADMIN , MANAGER

**Response** 200 OK :

```
{
  "success": true,
  "status": 200,
  "message": "Dashboard data retrieved successfully.",
  "data": {
    "totalAssets": 120,
    "byType": {
      "LAPTOP": 60,
      "MONITOR": 40,
      "ACCESSORY": 20
    },
    "byStatus": {
      "AVAILABLE": 15,
      "ASSIGNED": 90,
      "UNDER_REPAIR": 10,
      "DECOMMISSIONED": 5
    },
    "warrantyExpiringIn30Days": 8,
    "openConditionReports": 4
  }
}
```

---

### GET /reports/usage

Asset usage and allocation history report.

**Roles:** ADMIN , MANAGER

**Query params:** from (ISO date), to (ISO date), type , userId

**Response** 200 OK :

```
{
  "success": true,
  "status": 200,
  "message": "Usage report generated successfully.",
  "data": {
    "from": "2023-01-01",
    "to": "2023-12-31",
    "type": "all",
    "userId": null,
    "usage": [
      {
        "date": "2023-01-01",
        "asset": "LAPTOP",
        "status": "AVAILABLE",
        "userId": null,
        "usage": 15
      },
      {
        "date": "2023-01-01",
        "asset": "LAPTOP",
        "status": "ASSIGNED",
        "userId": null,
        "usage": 45
      },
      {
        "date": "2023-01-01",
        "asset": "LAPTOP",
        "status": "UNDER_REPAIR",
        "userId": null,
        "usage": 5
      },
      {
        "date": "2023-01-01",
        "asset": "LAPTOP",
        "status": "DECOMMISSIONED",
        "userId": null,
        "usage": 5
      },
      {
        "date": "2023-01-01",
        "asset": "MONITOR",
        "status": "AVAILABLE",
        "userId": null,
        "usage": 10
      },
      {
        "date": "2023-01-01",
        "asset": "MONITOR",
        "status": "ASSIGNED",
        "userId": null,
        "usage": 30
      },
      {
        "date": "2023-01-01",
        "asset": "MONITOR",
        "status": "UNDER_REPAIR",
        "userId": null,
        "usage": 5
      },
      {
        "date": "2023-01-01",
        "asset": "MONITOR",
        "status": "DECOMMISSIONED",
        "userId": null,
        "usage": 5
      },
      {
        "date": "2023-01-01",
        "asset": "ACCESSORY",
        "status": "AVAILABLE",
        "userId": null,
        "usage": 10
      },
      {
        "date": "2023-01-01",
        "asset": "ACCESSORY",
        "status": "ASSIGNED",
        "userId": null,
        "usage": 10
      },
      {
        "date": "2023-01-01",
        "asset": "ACCESSORY",
        "status": "UNDER_REPAIR",
        "userId": null,
        "usage": 5
      },
      {
        "date": "2023-01-01",
        "asset": "ACCESSORY",
        "status": "DECOMMISSIONED",
        "userId": null,
        "usage": 5
      }
    ]
  }
}
```

```
"data": {
  "period": { "from": "2024-01-01", "to": "2024-12-31" },
  "totalAllocations": 200,
  "averageAllocationDays": 45,
  "topUsers": [
    { "userId": "u-001", "name": "Jane Doe", "allocationsCount": 3 }
  ],
  "conditionReportsByMonth": [
    { "month": "2024-01", "count": 5 },
    { "month": "2024-02", "count": 3 }
  ]
}
```

---

## GET /reports/warranty-expiry

Assets with upcoming or past warranty expirations.

**Roles:** ADMIN , MANAGER

**Query params:** daysAhead (default 30)

**Response** 200 OK :

```
{
  "success": true,
  "status": 200,
  "message": "Warranty expiry report retrieved successfully.",
  "data": {
    "items": [
      {
        "id": "a-010",
        "brand": "Dell",
        "model": "XPS 15",
        "serialNumber": "SN-XPS-0042",
        "warrantyExpirationDate": "2025-07-01",
        "daysUntilExpiry": 14,
        "assignedTo": { "id": "u-001", "name": "Jane Doe" },
        "suggestedAction": "REASSIGN_AS_SPARE"
      }
    ]
  }
}
```

**Suggested actions:** REASSIGN\_AS\_SPARE , DECOMMISSION , RENEW\_WARRANTY

---

## Notifications

---

## GET /notifications

Get in-app notifications for the authenticated user.

Response **200 OK**:

```
{
  "success": true,
  "status": 200,
  "message": "Notifications retrieved successfully.",
  "data": {
    "items": [
      {
        "id": "n-001",
        "type": "WARRANTY_EXPIRY",
        "message": "Your laptop (Dell XPS 15, SN-XPS-0042) warranty expires in 14",
        "read": false,
        "createdAt": "2025-06-01T08:00:00Z"
      }
    ]
  }
}
```

---

## PATCH /notifications/{id}/read

Mark a notification as read.

Response **200 OK**:

```
{
  "success": true,
  "status": 200,
  "message": "Notification marked as read.",
  "data": {
    "id": "n-001",
    "read": true
  }
}
```

---

## GET /notifications/preferences

Get notification preferences for the current user.

Response **200 OK**:

```
{
  "success": true,
  "status": 200,
  "message": "Notification preferences retrieved successfully.",
  "data": {
    "warrantyExpiryAlerts": true,
    "lowStockAlerts": true,
    "emailNotifications": true,
    "daysBeforeWarrantyAlert": 30
  }
}
```

---

## PUT /notifications/preferences

Update notification preferences.

### Request:

```
{
  "warrantyExpiryAlerts": true,
  "lowStockAlerts": false,
  "emailNotifications": true,
  "daysBeforeWarrantyAlert": 14
}
```

### Response 200 OK :

```
{
  "success": true,
  "status": 200,
  "message": "Notification preferences updated successfully.",
  "data": {
    "warrantyExpiryAlerts": true,
    "lowStockAlerts": false,
    "emailNotifications": true,
    "daysBeforeWarrantyAlert": 14
  }
}
```

---

## Role Permission Summary

| Endpoint Group        | DEVELOPER                                                                           | MANAGER                                                                             | ADMIN                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Auth (register/login) |  |  |  |

|                          |   |   |   |
|--------------------------|---|---|---|
| Auth (register/login)    | ✓ | ✓ | ✓ |
| View own profile         | ✓ | ✓ | ✓ |
| View all users           | ✗ | ✓ | ✓ |
| Manage user roles        | ✗ | ✗ | ✓ |
| View own assets          | ✓ | ✓ | ✓ |
| View all assets          | ✗ | ✓ | ✓ |
| Create/update assets     | ✗ | ✓ | ✓ |
| Delete assets            | ✗ | ✗ | ✓ |
| Allocate/return assets   | ✗ | ✓ | ✓ |
| Submit condition reports | ✓ | ✓ | ✓ |
| Manage condition reports | ✗ | ✓ | ✓ |
| View reports & dashboard | ✗ | ✓ | ✓ |
| Manage notifications     | ✓ | ✓ | ✓ |

## Endpoint Validations

This section lists validation rules that each API endpoint must perform. Use `400 Bad Request` for syntactic issues (invalid JSON) and `422 Validation Error` for semantic/domain validation problems. Map DB uniqueness/constraint violations to `409 conflict` where appropriate.

### POST `/auth/register`

- Body: `name` (string, 1..120), `email` (valid email format, lowercase), `password` (min 8 chars, must include upper, lower, number, special)
- Enforce: `email` unique among active users → `409 EMAIL_CONFLICT`.
- Return `422` with `error.details` listing invalid fields for any broken rule.

### POST `/auth/login`

- Body: `email` (valid email), `password` (non-empty)
- Return `401 UNAUTHORIZED` for invalid credentials; do not reveal which field failed.

### GET `/users`

- Query params: `page` (int  $\geq 0$ ), `size` (int 1..100), `role` (must be one of `ADMIN` |

MANAGER|DEVELOPER if provided), `q` (optional free-text search)

- Search vector includes: `name`, `email`, `id` (uses PostgreSQL trigram similarity + case-insensitive substring matching; threshold 0.3)
- When `q` is provided with `role`, both filters apply (AND logic)
- Return `422` on invalid pagination or invalid role.

## GET `/users/{id}`

- Path param: `id` must be a valid UUID; otherwise `400`.

## PATCH `/users/{id}/role`

- Body: `role` required and must be in `user_role` enum.
- Only `ADMIN` may call; enforce `403` for others.
- Disallow removing the last `ADMIN` (return `409` or `422` with code `LAST_ADMIN_PROTECTION`).

## DELETE `/users/{id}`

- Path param: `id` UUID. Reject attempts to delete oneself with `422`.
- If user has active allocations ( `asset_allocations` with `returned_at IS NULL` ), return `409 USER_HAS_ACTIVE_ALLOCATIONS`.

## GET `/assets`

- Query params: `page` (int  $\geq 0$ ), `size` (int 1..100), `type`  $\in$  `asset_type` (if provided), `status`  $\in$  `asset_status` (if provided), `assignedUserId` must be UUID if present, `warrantyExpiresBefore` must be valid ISO date (if provided), `q` (optional free-text search across all asset fields: id, type, brand, model, serial number, status)
- Search vector includes: `id`, `type`, `brand`, `model`, `serialNumber`, `status` (uses PostgreSQL trigram similarity + case-insensitive substring matching; threshold 0.3)
- When `q` is provided with `type` / `status`, all filters apply (AND logic)
- Additional filters ( `assignedUserId`, `warrantyExpiresBefore` ) applied post-search
- Pagination: same `page` / `size` bounds as users.

## POST `/assets`

- Body required fields: `type`  $\in$  `asset_type`, `brand` (non-empty, max 80), `model` (non-empty, max 120), `serialNumber` (non-empty, max 100), `purchaseDate` (ISO date,  $\leq$  today), `warrantyExpirationDate` (ISO date,  $\geq$  `purchaseDate`).
- Enforce unique `serialNumber` among non-deleted assets  $\rightarrow$  `409 SERIAL_NUMBER_CONFLICT`.

## Allocation endpoints (assign/return)

- Assign: ensure `asset_id` exists, `status = AVAILABLE`, `assigned_user_id` is `NULL`, target user exists and is active; perform assignment inside a DB transaction and map unique-index violation to `409 ASSET_ALREADY_ASSIGNED`.
- Return: ensure an active allocation exists; otherwise `409 NO_ACTIVE_ALLOCATION`.

## Condition reports

- POST `/condition-reports`: `assetId` UUID must exist and be active, `issue` (min 10 chars), `severity`  $\in$  `report_severity`.
- PATCH status transitions: validate allowed transitions, require `resolution`, `resolved_by`, and `resolved_at` for `RESOLVED`.

## Notifications & Preferences

- Validate `days_before_warranty_alert`  $\in$  `[1..365]`; boolean flags required and default to `true` / `false` as documented.

## General rules

- All endpoints that accept UUIDs must validate UUID format and reject malformed IDs with `400 BAD_REQUEST`.
- Enforce `Content-Type: application/json` for JSON endpoints; return `415` if unspecified/incorrect.
- All authenticated endpoints require a valid JWT; a missing/expired token returns `401 UNAUTHORIZED`.
- Enforce role-based access control per-endpoint and return `403 FORBIDDEN` when the caller lacks the role